

Лабораторная работа №1 Динамические массивы

Цель работы:

- освоить описание динамических массивов;
- изучить обработку элементов динамических массивов.

Содержание лабораторной работы

Имя массива как указатель

Пример 1. Используя имя массива как указатель, и применяя адресную арифметику выполнить задание. Дан одномерный массив, состоящий из N целочисленных элементов.

- ввести массив с клавиатуры;
- найти максимальный отрицательный элемент;
- вычислить сумму отрицательных элементов массива;
- вывести положительные элементы на экран.

```
#include <iostream>
using namespace std;
void main() {
    setlocale(LC_ALL, "Russian");
    int x[100];
    int n, sum = 0;
    cin >> n;
    int maxotr = -2147483647;
    for (int i = 0; i < n; i++) {
        cin >> *(x + i);
    }
    for (int i = 0; i < n; i++) {
        if (*(x + i) < 0) {
            if (*(x + i) > maxotr)
                maxotr = *(x + i);
            sum += *(x + i);
        }
    }
    if (maxotr != -2147483647)
        cout << endl << "Максимальное отрицательное число: " << maxotr;
    else
        cout << endl << "Нет отрицательных";
    cout << endl << "Сумма отрицательных чисел: " << sum << endl;
    cout << "Положительные эл-ты массива: ";
    for (int i = 0; i < n; i++) {
        if (*(x + i) > 0)
            cout << *(x + i) << " ";
    }
    cout << endl;
}
```

Динамические массивы

Пример 2. Написать программу, в соответствии с заданием из предыдущего пункта с использованием динамических массивов, вводя размер массива с клавиатуры

```
#include <iostream>
using namespace std;
void main() {
    setlocale(LC_ALL, "Russian");
    int n, sum = 0;
```

```

cin >> n;
int *x = new int[n];
int maxotr = -2147483647;
for (int i = 0; i < n; i++) {
    cin >> *(x + i);
}
for (int i = 0; i < n; i++) {
    if (*(x + i) < 0) {
        if (*(x + i) > maxotr)
            maxotr = *(x + i);
        sum += *(x + i);
    }
}
if (maxotr != -2147483647)
    cout << endl << "Максимальное отрицательное число: " << maxotr;
else    cout << endl << "Нет отрицательных";
cout << endl << "Сумма отрицательных чисел : " << sum << endl;
cout << "Положительные эл-ты массива : ";
for (int i = 0; i < n; i++) {
    if (*(x + i) > 0)
        cout << *(x + i) << " ";
}
cout << endl;
delete[] x;
}

```

Пример 3. Найти сумму элементов в динамическом массиве. Размерность массива ввести с клавиатуры. *Использовать различные варианты при обращении к массиву.*

```

#include <iostream>
using namespace std;
void main()
{
    setlocale(LC_ALL, "Russian");
    int N;                                //введем размерность массива
    cout << "Введите размерность массива: ";
    cin >> N;
    int *a = new int[N];                  //создаем массив динамически
    for (int i = 0; i < N; i++)            //введем элементы массива с клавиатуры
    {
        cout << "Введите " << i << "-й элемент массива: ";
        cin >> a[i];
    }
    int s = 0;                            //объявим переменную для хранения суммы элементов
    for (int i = 0; i < N; i++)            //просуммируем элементы массива
        s += *(a + i);
    cout << "a:";                          //выведем на экран элементы массива и их сумму
    for (int i = 0; i < N; i++)
        cout << " " << i[a];
    cout << endl << "Сумма элементов: " << s << endl;
    delete[] a;                          //очищаем выделенную память
}

```

Пример 4. Удалить из массива все четные элементы

```
#include <iostream>
using namespace std;
int form(int a[100])    // заполнение массива
{
    int n;
    cout << "\nEnter n";
    cin >> n;
    for (int i = 0; i < n; i++)
        a[i] = rand() % 100;
    return n;
}
void print(int a[100], int n)    // вывод массива
{
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << "\n";
}
void Dell(int a[100], int&n)    // удаление четных элементов
{
    int j = 0, i, b[100];
    for (i = 0; i < n; i++)
        if (a[i] % 2 != 0)
        {
            b[j] = a[i]; j++;
        }
    n = j;
    for (i = 0; i < n; i++) a[i] = b[i];
}
void main()
{
    int a[100];
    int n;
    n = form(a);
    print(a, n);
    Dell(a, n);
    print(a, n);
}
```

Задания лабораторной работы

1. Дан динамический массив. Если массив не симметричен, то заменить четные элементы значением их квадрата, а нечетные элементы, не содержащие в своей записи цифры семь, значением заданного элемента x.
2. Дан динамический массив. Заменить все простые элементы значением их индекса.
3. Дан динамический массив. Удалить все отрицательные двузначные элементы.
4. Дан динамический массив. Вставить после каждого отрицательного элемента его квадрат.
5. Дано два динамических массива. Получить новый динамический массив, содержащий все элементы первого массива, не входящие во второй массив.
6. Дано два динамических массива. Получить новый динамический массив, содержащий только положительные элементы первого массива, входящие во второй массив ровно два раза.

Задания для самостоятельной работы

1. Дан динамический массив. Если массив не отсортирован, то заменить отрицательные элементы значением их модуля, а положительные элементы, не содержащие в своей записи единиц и нулей, значением максимального элемента.
2. Дан динамический массив. Удалить все непустые элементы.
3. Дан динамический массив. Вставить до каждого числа, в записи которого ровно две двойки новый элемент со значением 22.
4. Дано два динамических массива. Получить новый динамический массив, содержащий все элементы первого массива по одному разу, входящие и во второй массив.
5. Дано два динамических массива. Получить новый динамический массив, содержащий только те двузначные элементы первого массива, которые входят во второй массив не менее двух раз.

Лабораторная работа №2 **Динамические матрицы**

Цель работы:

- освоить основные функции обработки динамических матриц.

```
int **matr;  
matr=new int*[4]; //выделяем память под массив указателей int* их n элементов  
for (int i=0;i<4;i++)matr[i]=new int[6]; //выделяем память под строки массива
```

Задания лабораторной работы

1. Создание матрицы для хранения данных о погоде за определенный период времени (структура: температура, влажность, давление). Динамическая матрица позволит легко добавлять новые данные и изменять размер матрицы в зависимости от количества дней, за которые собираются данные (строка – время, столбец – день). Найти дату с пиками и средними значениями температур, влажности и давления.
2. Создание программы для анализа финансовых данных. Динамическая матрица будет использоваться для хранения информации о доходах и расходах компании за определенный период времени. Размеры матрицы будут меняться в зависимости от количества месяцев, за которые собираются данные (строка – бизнес-процессы компании: производство товара1, производство товара 2, реклама, зарплата, расходные материалы и т.д., столбец – месяц). Рассчитать убыточные и прибыльные месяцы, определить средний доход и месяцы и бизнес-процессы со значениями выше среднего.

Лабораторная работа №3-4 **Комплексные задачи на обработку строк и структур**

Цель работы:

- освоить основные функции обработки структур;
- изучить обработку элементов строки.

Содержание лабораторной работы

Для того чтобы использовать объекты класса string, необходимо включить соответствующий заголовочный файл: `#include <string>`

Объявление переменной типа string осуществляется точно так же как и обычной переменной. Возможные варианты:

```
string s1; // переменная с именем s1 типа string  
string s2 = "This is a string variable"; // объявление с инициализацией
```

Использование переменной типа `string` с оператором присваивания

```
s1 = s2; // s1 = "This is a string variable"
```

```
s2 = "New text";
```

Длину строки возвращает функция класса `size()` (длина не включает завершающий нулевой символ).

```
cout << "Длина " << s2.size()
```

Следующее определение строки задает пустую строку:

```
string s1; // пустая строка
```

Для проверки пустоты строки можно сравнить ее длину с 0:

```
if ( ! s1.size() ) // правильно: пустая
```

Однако есть и специальный метод `empty()`, возвращающий `true` для пустой строки и `false` для непустой:

```
if ( s1.empty() ) // правильно: пустая
```

Для сравнения строк используется оператор сравнения (`==`):

```
if ( s1 == s2 )
```

Для того, чтобы скопировать одну строку в другую можно использовать операцию присваивания:

```
st3 = st2; // копируем st2 в st3
```

Для конкатенации строк используется операция сложения (+) или операция сложения с присваиванием (`+=`). Пусть даны две строки:

```
string s1="hello, ";
```

```
string s2="world\n" ;
```

```
string s3 = s1 + s2;
```

Если же следует добавить `s2` в конец `s1`, необходимо написать:

```
s1 += s2;
```

Создание нового типа `string` было обусловлено недостатками работы со строками символов, который демонстрировал тип `char*`. В сравнении с типом `char*` тип `string` имеет следующие основные преимущества:

- возможность обработки строк стандартными операторами C++ (`=`, `+`, `==`, `<>` и т.п.).

При использовании типа `char*` даже наиболее простые операции со строками выглядели сложно и требовали написания чрезмерного программного кода;

- обеспечение лучшей надежности (безопасности) программного кода. Например, при копировании строк, тип `string` обеспечивает соответствующие действия, которые могут возникнуть в случае, если строка-источник имеет больший размер, чем строка-приемник;

- обеспечение строки, как самостоятельного типа данных. Объявление типа `string` как строки обеспечивает непротиворечивость данных.

Основным недостатком типа `string` в сравнении с типом `char*`, является замедленная скорость обработки данных. Это связано с тем, что тип `string` – это, фактически, контейнерный класс. А работа с классом требует дополнительной реализации программного кода, который, в свою очередь занимает лишнее время.

Класс `string` удобен тем, что позволяет удобно манипулировать строками, используя стандартные (перегруженные) операторы.

С объектами класса `string` можно использовать нижеследующие операторы

`=` – присваивание

`+` – конкатенация (объединение строк)

`+=` – присваивание с конкатенацией

`==` – равенство

`!=` – неравенство

`<` – меньше

`<=` – меньше или равно

`>` – больше

`>=` – больше или равно

[] – индексация

Пример 1. Тип string, операции над строками

```
string s1 = "s-1";
```

```
string s2 = "s-2";
```

```
string s3;
```

```
bool b;
```

// операция '=' (присваивание строк)

```
s3 = s1; // s3 = "s-1"
```

// операция '+' - конкатенация строк

```
s3 = s3 + s2; // s3 = "s-1s-2"
```

// операция '+= ' - присваивание с конкатенацией

```
s3 = "s-3";
```

```
s3 += "abc"; // s3 = "s-3abc"
```

// операция '==' - сравнение строк

```
b = s2==s1; // b = false
```

```
b = s2=="s-2"; // b = true
```

// операция '!=' - сравнение строк (не равно)

```
s1 = "s1";
```

```
s2 = "s2";
```

```
b = s1 != s2; // b = true
```

// операции '<' и '>' - сравнение строк

```
s1 = "abcd";
```

```
s2 = "de";
```

```
b = s1 > s2; // b = false
```

```
b = s1 < s2; // b = true
```

// операции '<=' и '>=' - сравнение строк (меньше или равно, больше или равно)

```
s1 = "abcd";
```

```
s2 = "ab";
```

```
b = s1 >= s2; // b = true
```

```
b = s1 <= s2; // b = false
```

```
b = s2 >= "ab"; // b = true
```

// операция [] - индексация

```
char c;
```

```
s1 = "abcd";
```

```
c = s1[2]; // c = 'c'
```

```
c = s1[0]; // c = 'a'
```

Как и любой класс, класс string имеет ряд конструкторов. Основные из них следующие:

```
string();
```

```
string(const char * str);
```

```
string(const string & str);
```

Рассмотрим примеры инициализации с помощью конструкторов

```
string s1("Hello!"); // инициализация - конструктор string(const char * str)
```

```
string s2 = "Hello!"; // инициализация - конструктор string(const char * str)
```

```
char * ps = "Hello";
```

```
string s3(ps);           // инициализация
string s4(s3);           // инициализация - конструктор string(const string & str)
string s5;               // инициализация - конструктор string()
```

Чтобы присвоить одну строку другой, можно применить один из двух методов:

- использовать оператор присваивания “=”;
- использовать функцию assign() из класса string.

Функция assign() имеет несколько перегруженных реализаций.

Первый вариант – это вызов функции без параметров

```
string &assign(void);
```

В этом случае происходит простое присваивание одной строки другой.

Второй вариант позволяет копировать заданное количество символов из строки:

```
string &assign(const string & s, size_type st, size_type num);
```

где

s – объект, из которого берется исходная строка;

st – индекс (позиция) в строке, из которой начинается копирование num символов;

num – количество символов, которые нужно скопировать из позиции st;

size_type – порядковый тип данных.

Третий вариант функции assign() копирует в вызывающий объект первые num символов строки s:

```
string &assign(const char * s, size_type num);
```

где

s – строка, которая завершается символом ‘\0’;

num – количество символов, которые копируются в вызывающий объект. Копируются первые num символов из строки s.

Пример 2. Присваивание строк, функция assign()

```
string s1 = "bestprog.net";
```

```
string s2;
```

```
string s3;
```

```
char * ps = "bestprog.net";
```

```
s3 = s1;           // s3 = "bestprog.net"
```

```
s2.assign(s1);     // s2 = "bestprog.net"
```

```
s2.assign(s1, 0, 4); // s2 = "best"
```

```
s2.assign(ps, 8);  // s2 = "bestprog"
```

Объединение строк. Функция append()

Для объединения строк используется функция append(). Для добавления строк также можно использовать операцию ‘+’, например:

```
string s1;
```

```
string s2;
```

```
s1 = "abc";
```

```
s2 = "def";
```

```
s1 = s1 + s2;      // s1 = "abcdef"
```

Функция append() хорошо подходит, если нужно добавлять часть строки. Она имеет следующие варианты реализации:

```
string &append(const string & s, size_type start);
```

```
string &append(const char * s, size_type num);
```

В первом варианте реализации функция получает ссылку на строчный объект s, который добавляется к вызывающему объекту. Во втором варианте реализации функция получает указатель на строку типа const char *, которая завершается символом ‘\0’.

Пример 3. Демонстрация работы функции append().

```
string s1 = "abcdef";
s2 = "1234567890";
append(s2, 3, 4);           // s1 = "abcdef4567"
```

```
char *ps = "1234567890";
s1 = "abcdef";
s1.append(ps, 3);           // s1 = "abcdef123"
```

Вставка символов в строке. Функция insert()

Чтобы вставить одну строку в заданную позицию другой строки нужно использовать функцию insert(), которая имеет несколько вариантов реализации.

Первый вариант функции позволяет вставить полностью всю строку s в заданную позицию start вызывающей строки (вызывающего объекта):

```
string &insert(size_type start, const string &s);
```

Второй вариант функции позволяет вставить часть (параметры insStart, num) строки s в заданную позицию start вызывающей строки:

```
string &insert(size_type start, const string &s, size_type insStart, size_type num);
```

В вышеприведенных функциях:

s – строка, которая вставляется в вызывающую строку;

start – позиция в вызывающей строке, из которой осуществляется вставка строки s;

insStart – позиция в строке s, из которой происходит вставка;

num – количество символов в строке s, которые вставляются с позиции insStart.

```
string s1 = "abcdef";
string s2 = "1234567890";
s1.insert(3, s2);           // s1 = "abc"+"1234567890"+"def"="abc1234567890def"
s2.insert(2, s1, 1, 3);     // s2 = "12bcd34567890"
```

Замена символов в строке. Функция replace()

Функция replace() выполняет замену символов в вызывающей строке. Функция имеет следующие варианты реализации:

```
string &replace(size_type start, size_type num, const string &s);
```

```
string &replace(size_type start, size_type num, const string &s, size_type replStart, size_type replNum);
```

В первом варианте реализации вызывающая строка заменяется строкой s. Есть возможность задать позицию (start) и количество символов (num) в вызывающей строке, которые нужно заменить строкой s.

Второй вариант функции replace() отличается от первого тем, что позволяет заменять вызывающую строку только частью строки s. В этом случае задаются два дополнительных параметра: позиция replStart и количество символов в строке s, которые образуют подстроку, которая заменяет вызывающую строку.

Пример 4. Замена символов, функция replace()

```
string s1 = "abcdef";
string s2 = "1234567890";
```

```
s2.replace(2, 4, s1);       // s2 = "12abcdef7890"
```

```
s2 = "1234567890";
```

```
s2.replace(3, 2, s1);       // s2 = "123abcdef67890"
```

```
s2 = "1234567890";
```

```
s2.replace(5, 1, s1);       // s2 = "12345abcdef7890"
```

```
s2 = "1234567890";
```

```
s2.replace(5, 1, s1, 2, 3);           // s2 = "12345cde7890"

s2 = "1234567890";
s2.replace(4, 2, s1, 0, 4);          // s2 = "1234abcd7890"
```

Удаление заданного количества символов из строки. Функция `erase()`.

Для удаления символов из вызывающей строки используется функция `erase()`:

```
string & erase(size_type index=0, size_type num = npos);
```

где

`index` – индекс (позиция), начиная из которой нужно удалить символы в вызывающей строке;

`num` – количество символов, которые удаляются.

Пример 5.

```
string s = "01234567890";
s.erase(3, 5);           // s = "012890"
s = "01234567890";
s.erase();               // s = ""
```

Поиск символа в строке. Функции `find()` и `rfind()`.

В классе `string` поиск строки в подстроке можно делать двумя способами, которые отличаются направлением поиска:

- путем просмотра строки от начала до конца с помощью функции `find()`;
- путем просмотра строки от конца к началу функцией `rfind()`.

Прототип функции `find()` имеет вид:

```
size_type find(const string &s, size_type start = 0) const;
```

где

`s` – подстрока, которая ищется в строке, что вызывает данную функцию. Функция осуществляет поиск первого вхождения строки `s`. Если подстрока `s` найдена в строке, что вызвала данную функцию, тогда возвращается позиция первого вхождения. В противном случае возвращается `-1`;

`start` – позиция, из которой осуществляется поиск.

Прототип функции `rfind()` имеет вид:

```
size_type rfind(const string &s, size_type start = npos) const;
```

где

`s` – подстрока, которая ищется в вызывающей строке. Поиск подстроки в строке осуществляется от конца к началу. Если подстрока `s` найдена в вызывающей строке, то функция возвращает позицию первого вхождения. В противном случае функция возвращает `-1`;

`npos` – позиция последнего символа вызывающей строки;

`start` – позиция, из которой осуществляется поиск.

Пример 6. Фрагмент кода, который демонстрирует результат работы функции `find()`

```
string s1 = "01234567890";
string s2 = "345";
string s3 = "abcd";
int pos;

pos = s1.find(s2);           // pos = 3
pos = s1.find(s2, 1);        // pos = 3
pos = s1.find("jklmn", 0);    // pos = -1
pos = s1.find(s3);           // pos = -1
pos = s2.find(s1);           // pos = -1
```

Пример 7. Демонстрация работы функции `rfind()`.

```

string s1 = "01234567890";
string s2 = "345";
string s3 = "abcd";
string s4 = "abcd---abcd";
int pos;

pos = s1.rfind(s2);           // pos = 3
pos = s1.rfind(s2, 12);      // pos = 3
pos = s1.rfind(s2, 3);       // pos = 3
pos = s1.rfind(s2, 2);       // pos = -1
pos = s2.rfind(s1);          // pos = -1
pos = s1.rfind(s3, 0);       // pos = -1

// разница между функциями find() и rfind()
pos = s4.rfind(s3);          // pos = 7
pos = s4.find(s3);           // pos = 0

```

Сравнение частей строк. Функция compare().

Поскольку тип string является классом, то, чтобы сравнить две строки между собой можно использовать операцию '=='. Если две строки одинаковы, то результат сравнения будет true. В противном случае, результат сравнения будет false.

Но если нужно сравнить часть одной строки с другой, то для этого предусмотрена функция compare().

Прототип функции compare():

```
int compare(size_type start, size_type num, const string &s) const;
```

где

s – строка, которая сравнивается с вызывающей строкой;

start – позиция (индекс) в строке s, из которой начинается просмотр символов строки для сравнения;

num – количество символов в строке s, которые сравниваются с вызывающей строкой.

Функция работает следующим образом. Если вызывающая строка меньше строки s, то функция возвращает -1 (отрицательное значение). Если вызывающая строка больше строки s, функция возвращает 1 (положительное значение). Если две строки равны, функция возвращает 0.

Пример 8. Демонстрация работы функции compare():

```

string s1 = "012345";
string s2 = "0123456789";
int res;

res = s1.compare(s2);           // res = -1
res = s1.compare("33333");      // res = -1
res = s1.compare("012345");     // res = 0
res = s1.compare("345");        // res = -1
res = s1.compare(0, 5, s2);     // res = -1
res = s2.compare(0, 5, s1);     // res = -1
res = s1.compare(0, 5, "012345"); // res = -1
res = s2.compare(s1);          // res = 1
res = s2.compare("456");        // res = -1
res = s2.compare("000000");     // res = 1

```

Получение строки с символом конца строки '\0' (char *). Функция c_str().

Чтобы получить строку, которая заканчивается символом `'\0'` используется функция `c_str()`.

Прототип функции:

```
const char * c_str() const;
```

Функция объявлена с модификатором `const`. Это означает, что функция не может изменять вызывающий объект (строку).

Пример 9. Преобразование типа `string` в `const char *`.

```
string s = "abcdef";
```

```
const char * ps;
```

```
ps = s.c_str(); // ps = "abcdef"
```

Пример 10. Заменить все точки символами подчеркивания:

```
string str( "fa.disney.com" );
```

```
int size = str.size();
```

```
for ( int ix = 0; ix < size; ++ix )
```

```
    if ( str[ ix ] == '.' )
```

```
        str[ ix ] = '_';
```

Однако этот фрагмент можно заменить вызовом функции `replace()`:

```
replace( str.begin(), str.end(), '.', '_' );
```

Здесь используется еще одна модификация функции (от первого, до последнего символа, заменить один на другой).

Задания лабораторной работы

1. На вход программе подается последовательность цифр, заканчивающаяся точкой (другие символы, кроме цифр и точки, отсутствуют). Требуется написать программу, которая выводит цифры, встречающиеся во входной последовательности, в порядке увеличения частоты их встречаемости. Если какие-то цифры встречаются одинаковое число раз, они выводятся в порядке возрастания. Например, если исходная последовательность была такая:

123124456.

то результат должен быть следующий: **356124**

2. На вход программе подается последовательность символов, заканчивающаяся нулем. Ноль в этой последовательности единственный, среди символов обязательно есть другие десятичные цифры. Требуется написать программу, которая составляет из этих цифр число-палиндром максимальной длины (которое читается одинаково слева направо и справа налево). Если таких чисел несколько, нужно вывести минимальное из них. Нулей в числе быть не должно (ноль – признак окончания ввода). Все имеющиеся цифры использовать не обязательно, но количество цифр в ответе должно быть максимально возможным. Например, если исходная последовательность была такая:

for i:=99921 downto 20

то результат должен быть следующий:

29192

3. На вход программе подается предложение (посимвольно) на английском языке, заканчивающееся точкой. Требуется написать программу, которая определяет, можно ли переставить английские буквы этого предложения (не учитывая остальные символы) так, чтобы получить палиндром – слово, которое читается одинаково слева направо и справа налево. Строчные и прописные буквы не различаются. Если это сделать нельзя, программа должна вывести на экран слово «Нет», а если можно – слово «Да» и в следующей строке искомое

слово прописными буквами. Если таких слов несколько, нужно вывести *последнее* в алфавитном порядке слово. Например, если исходная последовательность была такая:

Deed daad N.

то результат должен быть следующий:

Да

EDDANADDE

4. На вход программе подается строка (длиной не более 200 символов, посимвольно), в которой нужно зашифровать все английские слова (словом называется непрерывная последовательность английских букв, слова друга от друга отделяются любыми другими символами, длина слова не превышает 20 символов). Строка заканчивается символом #, других символов # в строке нет. Каждое слово зашифровано с помощью циклического сдвига на длину этого слова. Например, если длина слова равна K, каждая буква в слове заменяется на букву, стоящую в английском алфавите на K букв дальше (алфавит считается циклическим, то есть, за буквой Z стоит буква A). Строчные буквы при этом остаются строчными, а прописные – прописными. Символы, не являющиеся английскими буквами, не изменяются.

Требуется написать программу, которая будет выводить на экран текст зашифрованного сообщения. Например, если исходный текст был таким:

Day, mice. "Year" is a mistake#

то результат шифровки должен быть следующий:

Gdb, qmgi. "Ciev" ku b tpzahrl#

5. Описать структуру MARSH, содержащую поля:

- Название начального пункта маршрута
- Название конечного пункта маршрута
- Номер маршрута

Ввести не менее 10 записей, упорядочить их по возрастанию номера маршрута. Вывести информацию о маршрутах, которые начинаются или заканчиваются в заданном с клавиатуры пункте.

6. Описать структуру NOTE, содержащую поля:

- Фамилия, имя
- Номер телефона
- День рождения (три числа)

Ввести массив структур, упорядочить данные по датам рождения. Вывести информацию о людях, чьи телефоны не содержат нулей и оканчиваются на 33. Вывести номера телефонов людей, фамилии которых начинаются на «А» и «И».

Лабораторная работа №5

Простые классы

Цель работы:

- освоить конструирование простого класса;

Задания лабораторной работы

1. Создать класс "Студент", который будет иметь поля: имя, фамилия, возраст, группа, средний балл. Необходимо реализовать методы для вычисления среднего балла студента, а также конструкторы для инициализации объекта класса различными способами (например, заданием имени, фамилии и группы).

2. Создать класс "Книга", который будет иметь поля: название, автор, издательство, год издания, количество страниц. Необходимо реализовать методы для вывода информации о книге, а также конструкторы для инициализации объекта класса различными способами (например, заданием названия и автора).

3. Создать класс "Товар", который будет иметь поля: название, цена, количество на складе. Необходимо реализовать методы для изменения количества товара на складе, а также конструкторы для инициализации объекта класса различными способами (например, заданием названия и цены).

4. Создать класс "Автомобиль", который будет иметь поля: марка, модель, год выпуска, цвет, количество дверей. Необходимо реализовать методы для вывода информации об автомобиле, а также конструкторы для инициализации объекта класса различными способами (например, заданием марки и модели).

5. Создать класс "Человек", который будет иметь поля: имя, фамилия, возраст, пол, рост, вес. Необходимо реализовать методы для вычисления индекса массы тела человека, а также конструкторы для инициализации объекта класса различными способами (например, заданием имени и возраста).

6. Создать класс "Заказ", который будет иметь поля: номер заказа, дата заказа, список товаров, общая стоимость заказа. Необходимо реализовать методы для добавления товаров в заказ, а также конструкторы для инициализации объекта класса различными способами (например, заданием номера заказа и даты).

Лабораторная работа №6-7 **Однонаправленные списки**

Цель работы:

- освоить построение однонаправленного динамического списка;
- изучить конструкцию вывода элементов списка;
- изучить алгоритмы обработки элементов списка;
- изучить алгоритмы вставки и удаления элементов списка.

Содержание лабораторной работы

Описание элемента списка выглядит следующим образом:

```
struct имя_типа
{
    информационное поле;
    адресное поле;
};
```

Информационное поле – это поле любого, ранее объявленного или стандартного, типа; адресное поле – это указатель на объект того же типа, что и определяемая структура, в него записывается адрес следующего элемента списка. Информационных полей может быть несколько.

```
struct Node
{int info;                //информационное поле
Node*next;               //адресное поле
};
```

Над списками можно выполнять следующие операции:

- начальное формирование списка (создание первого элемента);
- добавление элемента в конец списка;
- добавление элемента в начало списка;
- удаление элемента с заданным номером;

- чтение элемента с заданным ключом;
- вставка элемента в заданное место списка (до или после элемента с заданным ключом);
- упорядочивание списка по ключу

и др.

Пример 1. Создание и вывод элементов однонаправленного списка

```
#include <iostream.h>
#include<string.h>
struct list
{int info;
list*next;
};
list* make_list()                //формирование списка до введения нуля
{
    list* beg = new(list);
    list*r, *p;
    int x;
    cin >> x;
    beg->info = x;
    p = beg;
    while (x != 0)
    {
        cin >> x;
        if (x != 0) {
            r = new(list);
            r->info = x;
            r->next = NULL;
            p->next = r;
            p = r;
        }
    }
    return beg;
}

void print(list*beg)
{
    list*p=beg;
    while (p!=NULL)
    {
        cout<<"\nElem:"<<p->info<<" ";
        p=p->next;
    }
}

void main()
{
    list*beg=make_list();
```

```
print(beg);
}
```

Пример 2. Удаление из однонаправленного списка элемента со значением k

```
list*del(list*&beg, int k)
{
    list*p = beg;
    list *r;
    while (p->next != NULL)
    {
        if (p->next->info == k)
        {
            r = p->next;
            if (r->next!=NULL)
            {
                p->next = r->next;
                delete[]r;
            }
            else {
                p->next = NULL;
                delete[]r; break;
            }
        }
        p = p->next;
    }
    if (beg->info == k) {
        p = beg;
        beg = beg->next;
        delete[]p;
    }
    return beg;
}

void main()
{
    int n;
    list*beg = make_list();
    cin >> n;
    del(beg, n);
    print(beg);
}
```

Пример 3. Вставить новый элемент со значением z ПОСЛЕ элемента со значением k.

```
list*vst(list*&beg, int k, int z)
{
    list*p = beg;
    list *r;
```

```

while (p != NULL)
{
    if (p->info == k)
    {
        r = new(list);
        r->info=z;
        if (p->next!=NULL)
        { r->next = p->next;
          p->next = r;
        }
        else {
            r->next = NULL;
            p->next=r;
        }
    }
    p = p->next;
}
return beg;
}

```

Задания лабораторной работы

1. Построить однонаправленный список целых чисел. Найти сумму квадратов нечетных элементов.
2. Заменить отрицательные элементы однонаправленного списка, за которыми следуют нечетные элементы, модулями этих отрицательных чисел.
3. Найти количество кратных 5 элементов, значение которых превосходит среднее арифметическое положительных элементов списка.
4. Заменить некратные 3 элементы списка, суммой нечетных элементов.
5. Проверить список на упорядоченность по возрастанию.
6. Увеличить двузначные положительные, за которыми следуют отрицательные на значение наибольшего элемента списка.
7. Удалить все положительные элементы, за которыми следуют четные.
8. Удалить все кратные 5 элементы, до которых идет отрицательный элемент.
9. Удалить все двузначные элементы, расположенные между равными элементами.
10. Вставить новый элемент между двумя соседними четными элементами.
11. Вставить элемент равный 5 до каждого нечетного элемента списка.
12. Вставить элемент со значением наименьшего четного элемента после каждого элемента, оканчивающегося на 13.

Задания для самостоятельной работы

1. Построить однонаправленный список целых чисел. Заменить элементы, оканчивающиеся на 2, суммой квадратов отрицательных элементов списка.
2. Построить однонаправленный список целых чисел. Найти произведение квадратов элементов, оканчивающихся на 22.
3. Верно ли, что однонаправленный список является знакопеременным.
4. Удалить из однонаправленного списка все элементы, у которых последняя и предпоследняя цифры равны.

5. Построить однонаправленный список целых чисел. Удалить все делящиеся на 8 элементы, перед которыми нет нечетного числа.
6. Вставить элемент со значением количества положительных элементов списка, до каждого трехзначного числа.

Лабораторная работа №8-9 **Однонаправленные списки. Сложная обработка**

Цель работы:

- освоить обработку элементов списка, включающих элементы со сложными свойствами (использование подпрограмм);
- изучить алгоритмы обработки нескольких списков;
- освоить построение списка по массиву, элементы которого обладают заданными свойствами.

Задания для лабораторной работы

1. Вставить минимальный элемент списка до каждого элемента с четной суммой цифр (функция – сумма цифр).
2. Удалить из списка все простые элементы.
3. Вставить между двумя совершенными элементами новый элемент со значением «11».
4. Дан одномерный массив. Записать в однонаправленный список только те элементы массива, в записи которых ровно две «2». Затем из списка удалить все элементы, оканчивающиеся на 2.
5. Дан однонаправленный список. Если в нем нет элементов, состоящих только из цифры 7 (функция), то построить другой однонаправленный список, включающий только те элементы, у которых ровно три цифры 7.
6. Дан однонаправленный список. Построить одномерный массив, включающий только те элементы, у которых ровно три делителя кратных 5 (функция). Затем проверить массив на возрастание элементов.
7. Дано два однонаправленных списка. Получить третий список, содержащий только положительные элементы исходных списков.
8. Дано два однонаправленных, упорядоченных по возрастанию, списка. Получить третий список, также упорядоченный по возрастанию.

Задания для самостоятельной работы

1. Заменить все элементы списка, являющиеся числами Фибоначчи, количеством отрицательных элементов списка.
2. Вставить элемент равный количеству некратных 3 элементов, после каждого двузначного элемента.
3. Удалить из списка все двузначные и трехзначные элементы, не содержащие в своей записи цифру «1».
4. Дан массив целых чисел. Построить однонаправленный список, содержащий только непростые элементы массива.
5. Дан однонаправленный список. Если не знакочередующийся, то построить другой однонаправленный список, включающий только отрицательные элементы с четным количеством цифр, а затем проверить его на знакочередование.
6. Дан однонаправленный список вещественных чисел. Записать в одномерный массив только те элементы списка, которые имеют одну цифру после запятой.
7. Дано два однонаправленных списка. Получить третий список, содержащий четные элементы первого списка, не входящие во второй список.

8. Дано два однонаправленных списка. Получить третий список, содержащий все различные числа исходных списков (удаление элементов не использовать).

Лабораторная работа №10-11

Двунаправленные списки

Цель работы:

- освоить построение двунаправленного динамического списка;
- изучить конструкцию вывода элементов списка;
- изучить алгоритмы обработки элементов списка;
- изучить алгоритмы вставки и удаления элементов списка;
- освоить обработку элементов списка, обладающих сложными свойствами (использование подпрограмм).

Содержание лабораторной работы

Пример 1. Построение двунаправленного списка.

```
#include "stdafx.h"
#include <iostream>
using namespace std;
struct list
{
    int info;
    list*next;
    list*pred;
};
void print(list*beg)
{
    list*p = beg;
    while (p != NULL)
    {
        cout << "\nElem:" << p->info<< " ";
        p = p->next;
    }
}
list* make_list()
{
    list* beg = new(list);
    list*r, *p;
    int x;
    cin >> x;
    beg->info = x;
    p = beg;
    while (x != 0)
    {
        cin >> x;
        if (x != 0) {
            r = new(list);
```

```

        r->info = x;
        r->next = NULL;
        p->next = r;
        r->pred = p;
        p = r;
    }
}
return beg;
}

```

Пример 2. Удаление элемента со значением k.

```

list*del(list*&beg, int k)
{
    list*p = beg->next;
    list *r;
    while (p!= NULL)
    {
        if (p->info == k)
        {
            if (p->next != NULL)
            {
                r = p;
                p->next->pred = p->pred;
                p->pred->next = p->next;
                delete[]r;
            }
            else {
                p->pred->next = NULL;
                delete[]p; break;
            }
        }
        p = p->next;
    }
    if (beg->info == k) {
        p = beg;
        beg = beg->next;
        beg->pred = NULL;
        delete[]p;
    }
    return beg;
}

```

Задания для лабораторной работы

1. Дан двунаправленный список. Найти значение наибольшего элемента.
2. Проверить является ли список симметричным.
3. Вычислить $x_1 * x_n + x_2 * x_{n-1} + \dots + x_n * x_1$.
4. Заменить все оказывающиеся на 5 элементы, значение последнего четного элемента (движение от последнего элемента).

5. Вычислить $\max(x_1 * x_n, x_2 * x_{n-1}, \dots, x_{n/2} * x_{n/2})$.

6. Удалить все простые (функция) элементы двунаправленного списка, расположенные между четными элементами.

7. Дан двунаправленный список. Удалить все кратные 5 элементы списка, до которых идет элемент, в записи которого нет нулей (функция).

8. Дан двунаправленный список. Вставить максимальный элемент списка перед каждым нечетным элементом.

9. Дан двунаправленный список. Вставить элемент равный 11, между двумя элементами, содержащими цифры «1».

10. Дан двунаправленный список. Если список не упорядочен по возрастанию, удалить из него все совершенные (функция) элементы, иначе вставить перед каждым элементом, у которого совпадают первая и последняя цифры (функция) новый элемент со значением 5.

Задания для самостоятельной работы

1. Дан двунаправленный список. Найти количество несимметричных пар, содержащих только четные элементы.

2. Удалить все несовершенные (функция) элементы, расположенные между элементами равными по модулю.

3. Дан двунаправленный список. Если список не знакопеременный, то удалить все отрицательные элементы, иначе удалить отрицательные элементы, после которых расположены элементы с последней цифрой, кратной трем.

4. Дан двунаправленный список. Вставить после каждого элемента с четной суммой цифр (функция) элемент, равный значению этой суммы.

5. Дан двунаправленный список. Вставить после каждого элемента, у которого ровно три делителя, элемент равный удвоенному минимальному значению списка.

6. Дан двунаправленный список. Если он несимметричен, то удалить все его положительные элементы. В противном случае удалить только простые элементы списка.

7. Дан двунаправленный список. Если он не содержит элементы кратные трем, то вставить после каждого четного числа элемент равный 33. В противном случае удалить элементы списка, кратные девяти.

Лабораторная работа №12-13

Двунаправленные списки. Сложная обработка

Цель работы:

- изучить алгоритмы обработки нескольких списков;
- освоить построение списка по массиву, элементы которого обладают заданными свойствами;
- освоить рекурсивную обработку элементов динамического списка;
- изучить построение кольцевого списка;
- изучить алгоритмы обработки кольцевых списков.

Задания для лабораторной работы

1. Построить двунаправленный список. Если он не упорядочен, то удалить в списке все несовершенные элементы.

2. Дан двунаправленный список символов. Построить однонаправленный список, содержащий только цифры исходного списка, после которых идет элемент со значением «?» или «*».

3. Дан двунаправленный список. Построить одномерный массив, включающий только элементы массива, состоящие из одинаковых цифр (функция). Затем найти в массиве индекс максимального элемента.

4. Дан двунаправленный кольцевой список. Найти количество четных пар идущих подряд элементов.

5. Дан двунаправленный кольцевой список. Если в нем более двух элементов кратных пяти, то удалить эти элементы.

6. Дан одномерный массив целых чисел. Построить двунаправленный список, содержащий только кратные 5 элементы массива. Затем в списке удалить все элементы, оканчивающиеся на 55.

7. Дано два однонаправленных списка. Построить двунаправленный список, содержащий элементы из первого списка, не входящие во второй список.

8. Дано два двунаправленных списка. Построить однонаправленный список, содержащий элементы из первого списка, которые больше минимального и меньше максимального элементов второго списка.

Следующие задания выполнить *рекурсивно*.

9. Дан однонаправленный список. Заменить все четные элементы на значение суммы нечетных элементов списка.

10. Дан однонаправленный список. Уменьшить все элементы кратные трем на минимальный элемент списка.

11. Дан однонаправленный список. Если после замены всех отрицательных на их модули в списке нет четных элементов, то найти произведение элементов, оканчивающихся на 1.

Задания для самостоятельной работы

1. Построить двунаправленный список. Если в нем все элементы положительные, то вставить после каждого элемента с максимальным значением новый элемент, информационная часть которого равна количеству элементов, в записи которых есть нули.

2. Дан двунаправленный список цифр. Построить однонаправленный список, содержащий только четные цифры исходного списка, расположенные между единицами.

3. Дано два двунаправленных списка. Построить однонаправленный список, содержащий только те элементы первого списка, которые входят во второй список ровно два раза.

4. Дан двунаправленный кольцевой список. Найти сумму отрицательных элементов, расположенных между равными.

5. Дан двунаправленный кольцевой список. Если в нем более нет заданного числа x , то вставить его (x) после каждого числа, оканчивающегося на 25.

Следующие задания выполнить *рекурсивно*.

6. Дано два однонаправленных списка. Построить двунаправленный список, содержащий кратные 5 элементы из первого списка, входящие и во второй список.

7. Дано два двунаправленных списка. Построить однонаправленный список, содержащий положительные элементы из первого списка, которые больше суммы нечетных элементов второго списка.

8. Дан двунаправленный список. Построить одномерный массив, включающий только элементы простые массива (функция). Затем найти в массиве количество максимальных элементов.

Следующие задания выполнить *рекурсивно*.

9. Дан однонаправленный список. Увеличить все элементы кратные пяти на значение количества отрицательных элементов списка.

10. Дан однонаправленный список. Если максимальный элемент списка меньше заданного числа x , то заменить все элементы равные x на максимальный элемент списка.

11. Дан однонаправленный список. Верно ли, что после замены элементов, кратных 4 на удвоенный минимальный элемент списка, количество четных элементов списка будет не больше трех.

Лабораторная работа №14-17

Деревья двоичного поиска

Цель работы:

- освоить построение дерева двоичного поиска;
- изучение алгоритмов рекурсивной обработки двоичного дерева (функции и процедуры);
- освоить построение дерева двоичного поиска, включающего поле «уровень»;
- изучить алгоритмы рекурсивной обработки уровней двоичного дерева.

Содержание лабораторной работы

Для моделирования корневых деревьев определим типы:

```
struct tree
{
    int info;           //информационное поле
    tree*L;
    tree*R;
};

void add(tree*t,int x)
{
    tree*q;
    if (x<t->info)
        if (t->L == NULL)
            { q = new(tree); q->info = x;
              q->L = NULL; q->R = NULL; t->L = q; }
        else add(t->L, x);
    else
        if (t->R == NULL) { q = new(tree); q->info = x;
                           q->L = NULL; q->R = NULL; t->R = q; }
        else add(t->R, x);
}

void main()
{
    int x;
    tree*beg = new(tree);
    cin >> x;
    beg->info = x; beg->L = NULL; beg->R = NULL;
    while (x!=0)
    {
```

```

    cin >> x;
    if (x != 0) add(beg, x);

```

```

}

```

Обход (вывод информационных полей) дерева.

```

void print(tree*p)
{
    if (p != NULL)
    {
        cout << "\nElem:" << p->info;
        cout << "\n-----\n";
        print(p->L);
        print(p->R);           //переход к следующему элементу
    }
}

```

Пример 1. Найти количество четных листьев дерева.

```

int Kolvo(tree*p)
{
    if (p != NULL)
    { if (p->L == NULL && p->R == NULL && p->info %2==0)
        return 1 + Kolvo(p->L) + Kolvo(p->R);
        else return Kolvo(p->L) + Kolvo(p->R);
    }
    else return 0;
}

void main()
{
    int x;
    tree*beg = new(tree);
    cin >> x;
    beg->info = x; beg->L = NULL; beg->R = NULL;
    while (x!=0)
    {
        cin >> x;
        if (x != 0) add(beg, x);
    }
    cout << "Kolvo learn=" << Kolvo(beg);
    print(beg);
}

```

Пример 2. Заменить отрицательные элементы дерева нулем.

```

void Zam(tree*p)
{
    if (p != NULL)
    {
        If (p->info < 0) p->info=0;
    }
}

```

```

        Zam(p->L);
        Zam(p->R);
    }
}

```

Задания для лабораторной работы

Л 14

1. Дано дерево двоичного поиска. Найти количество положительных элементов, не являющихся листьями.
2. Найти произведение делящихся на 4 элементов дерева, имеющих ровно одного сына.
3. Заменить отрицательные элементы дерева их модулями.
4. Увеличить некратные 3 элементы дерева вдвое.
5. Упорядочить вводимые элементы с помощью дерева по убыванию.
6. Найти сумму простых чисел, не являющихся листьями дерева.
7. Найти произведение последних цифр листьев дерева.

Л 15

8. Найти максимальный элемент дерева.
9. Увеличить в дереве все элементы, являющиеся листьями на значение количества элементов, оканчивающихся на 5.
10. Верно, ли что в дереве все элементы четные.
11. Верно, ли что в дереве нет кратных 5 листьев.
12. Верно ли, что в дереве хотя бы один лист четный.
13. Найти количество чисел Фибоначчи, имеющих ровно одного сына.
14. Найти произведение элементов дерева, у которых более трех четных делителей и только левый четный сын.

Л 16

15. Найти наибольшее количество уровней в дереве (глубину дерева).
16. Дан массив чисел. Отсортировать его с помощью дерева в порядке возрастания и в порядке убывания.
17. Дан массив чисел. Исключить из него все повторяющиеся элементы с помощью дерева.
18. Дан массив чисел. Исключить из него все четные элементы с помощью дерева и записать в новый массив в порядке убывания.
19. Удалить из дерева все четные элементы.
20. Построить дерево из элементов массива, исключая повторяющиеся четные элементы. А затем из дерева поместить в массив упорядоченную по убыванию последовательность чисел, оканчивающуюся на 4 и в другой массив все листья дерева.

Л 17

21. Дан однонаправленный список целых чисел. Построить дерево, состоящее из положительных элементов однонаправленного списка.
22. Дан однонаправленный список целых чисел. Построить дерево, состоящее из нечетных элементов однонаправленного списка. Элемент дерева должен содержать значение и порядковый номер значения в списке.
23. Дан двунаправленный список целых чисел. Построить дерево, состоящее из оканчивающихся на 2 элементов списка. Затем в дереве найти сумму всех элементов, не являющихся листьями.
24. Дано дерево двоичного поиска. Построить однонаправленный список, содержащий только листья дерева.

Задания для самостоятельной работы

Л 14

1. Найти количество листьев дерева, значение которых больше среднего арифметического всех элементов.
2. Заменить нечетные элементы дерева произведением листьев.
3. Уменьшить все двузначные элементы дерева на значение его минимального элемента.

Л 15

4. Определить: есть ли в дереве хотя бы один элемент, равный половине значения наибольшего элемента дерева.
5. Найти сумму четных элементов дерева, значение которых больше значения корня.
6. Есть ли в дереве хотя бы один элемент, принадлежащий интервалу [2;9].

Л 16

7. Дан массив чисел. Исключить из него все отрицательные элементы с помощью дерева.
8. Дан массив чисел. Исключить из него все повторяющиеся нечетные элементы с помощью дерева и записать в новый массив в порядке возрастания.

Л 17

9. Дан двунаправленный список. Построить дерево, включив в него все четные элементы, встречающиеся в списке более одного раза (в дереве эти элементы указываются один раз). Найти количество элементов дерева меньших среднего арифметического всех элементов дерева.
10. Дан однонаправленный список целых чисел. Построить дерево, состоящее из кратных 7 элементов однонаправленного списка. Элемент дерева должен содержать значение и сумму цифр числа из списка.
11. Дан двунаправленный список целых чисел. Построить дерево, состоящее из не оканчивающихся на 1 элементов двунаправленного списка. Элемент дерева должен содержать значение и количество цифр числа из списка.

Лабораторная работа №18-20 Обработка типизированных файлов

Цель работы:

- освоить описание текстовых файлов;
- освоить конструкцию создания файла;
- изучить типовые алгоритмы обработки элементов текстового файла;
- изучить механизм записи значений в новый файл.

Содержание лабораторной работы

Пример 1. Построить файл целых чисел. Записать все положительные числа из исходного файла в новый файл.

```
#include "stdafx.h"
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    setlocale(LC_ALL, "rus"); // корректное отображение Кириллицы
    int s;
    int n;
    ofstream f("f.txt");
    cin >> n;
    for (int i = 0; i < n; i++)
    {
```

```

        cin >> s;
        f << s << endl;
    }
    f.close();
    ifstream h("f.txt");
    ofstream g("g.txt");
    while (!h.eof())
    {
        h >> s;
        if (s >= 0 ) g << s << endl;
    }
    h.close();
    g.close();
    system("pause");
    return 0;
}

```

Пример 2. Является ли файл упорядоченным по возрастанию элементов

```

#include "stdafx.h"
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    setlocale(LC_ALL, "rus");
    int a, b;
    int n;
    bool flag = 1;
    ofstream f("f.txt");
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cin >> a;
        f << a << endl;
    }
    f.close();
    ifstream h("f.txt");
    h >> a;
    while (!h.eof())
    {
        h >> b;
        if (a >= b) flag = 0;
    }
    h.close();
    cout << flag;
    system("pause");
    return 0;
}

```

Пример 3. Записать в новый файл все элементы исходного, значения которых больше количества четных элементов файла.

```

#include "stdafx.h"
#include <iostream>
#include <fstream>
using namespace std;

```

```

int main()
{
    setlocale(LC_ALL, "rus");
    int a, b;
    int n, k=0;
    ofstream f("f.txt");
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cin >> a;
        f << a << endl;
    }
    f.close();
    ifstream h("f.txt");
    while (!h.eof())
    {
        h >> a;
        if (a % 2==0) k++;
    }
    h.close();
    ifstream f1("f.txt");
    ofstream g("g.txt");
    while (!f1.eof())
    {
        f1 >> a;
        if (a>k) g<<a<<endl;
    }
    f1.close();
    g.close();
    system("pause");
    return 0;
}

```

Пример 4. Дан файл целых чисел. Если файл упорядочен по возрастанию, то записать в новый файл все числа, оканчивающиеся на 11 и не равные минимальному.

```

#include "stdafx.h"
#include <iostream>
#include <fstream>
using namespace std;
int main()
{
    setlocale(LC_ALL, "rus");
    int a, b;
    int n, k = 0;
    bool flag = 1;
    ofstream f("f.txt");
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cin >> a;
        f << a << endl;
    }
    f.close();
    ifstream h("f.txt");

```

```

h >> a;
while (!h.eof())
{
    h >> b;
    if (a > b) flag=0;
}
h.close();
ifstream h1("f.txt");
h1 >> a;
while (!h1.eof())
{
    h1 >> b;
    if (b < a) a=b; // a=min
}
h.close();
if (flag == 1)
{
    ifstream f1("f.txt");
    ofstream g("g.txt");
    while (!f1.eof())
    {
        f1 >> b;
        if (a != b && b % 100 == 11) g << b << endl;
    }
    f1.close();
    g.close();
}
system("pause");
return 0;
}

```

Задания для лабораторной работы

Л 18

1. Дан файл целых чисел. Найти сумму чисел, не оканчивающихся на 5.
2. Дан файл целых чисел. Найти максимальный элемент файла.
3. Дан файл целых чисел. Найти произведение последних цифр чисел, значение которых не кратно 3.
4. Найти произведение порядковых номеров делящихся на 8 элементов файла.

Л 19

5. Дан файл целых чисел. Проверить является ли файл упорядоченным по убыванию.
6. Дан файл целых чисел. Если файл не упорядочен по возрастанию, то записать в новый файл числа, у которых последняя цифра равна предпоследней.
7. Дан файл целых чисел. Записать в новый файл квадраты чисел, значения которых больше среднего арифметического всех элементов файла.
8. Дан файл целых чисел. Записать в новый файл те числа, значение которых больше чем $2 \cdot \min$.

Л 20

9. Дан текстовый файл f. Добавить в его конец символы e, n, d.
10. Дан текстовый файл f. Подсчитать число вхождений в файл сочетаний «ab».
11. Дан текстовый файл. Преобразовать строки файла по правилу: если строка содержит буквы, то удалить их.
12. Дан текстовый файл. Преобразовать строки файла по правилу: если строка несимметрична, то заменить в ней все цифры на «12».

13. Дан текстовый файл. Если строка содержит хотя бы одно слово, состоящее только из букв «а», то записать в новый файл первое слово этой строки.

14. Дан текстовый файл. Записать в новый файл номера строк и количество слов каждой строки.

Задания для самостоятельной работы

Л 18

1. Дан файл целых чисел. Найти наименьший четный элемент файла.

2. Дан файл целых чисел. Записать в новый файл порядковые номера тех элементов исходного файла, сумма цифр которых четна.

3. Найти сумму порядковых номеров двузначных положительных элементов.

Л 19

4. Дан файл целых чисел. Записать в другой файл те элементы исходного файла, у которых ровно три четных делителя.

5. Дан файл целых чисел. Найти количество элементов файла, значение которых превосходит среднее арифметическое положительных элементов файла.

6. Дан файл целых чисел. Записать в новый файл квадраты всех чисел исходного файла, в записи которых есть цифра 1.

Л 20

7. Дан текстовый файл. Преобразовать строки файла по правилу: если строка содержит больше двух цифр, то между каждой парой букв вставить «-».

8. Дан текстовый файл. Преобразовать строки файла по правилу: если в строке букв больше, чем цифр, то из первой половины удалить все буквы, а из второй половины удалить все цифры. Преобразованные строки записать в новый файл.

9. Дан текстовый файл. Вывести те строки, в которых есть симметричные слова.

10. Дан текстовый файл. Записать в новый файл для каждой строки количество слов, содержащих ровно одну «*».

11. Дан текстовый файл. Записать в новый файл для каждой строки те слова, в которых нет цифр.

Лабораторная работа №21-22

Графы

Цель работы:

- освоить основные способы представления графа;
- освоить алгоритм Флойда;
- освоить алгоритм Дейкстры;
- освоить алгоритм поиска центра графа;
- освоить алгоритм Прима;
- освоить алгоритм Крускала;

Содержание лабораторной работы

Заполнение матрицы смежности (ввод графа)

// Инициализация матрицы связей

```
for (int i = 1; i <= SIZE; i++)
```

```
    for (int j = 1; j <= SIZE; j++)
```

```
    {
```

```
        a[i][j] = 1000;
```

```
    }
```

```
cout << "Ведите количество ребер графа ";
```

```
cin >> m;
```

```

for (int j = 1; j <= m; j++)
{
    cout << "i->j, вес: ";
    cin >> x >> y >> vv;
    a[x][y] = vv;           //для неориентированного графа a[y][x] = vv;
}

```

Алгоритм Дейкстры.

```

S={1};
for (i=2; i<=n; i++)
    D[i]=C[1][ i]; {инициализация D}
for (i=1; i<=n-1; i++)
    { Выбор из множества V\S такой вершины w,
      что значение D[w] минимально;
      добавить w к множеству S;
      for каждая вершина v из множества V\S
          D[v]=min(D[v], D[w] + C[w][v])
    }

```

Алгоритм нахождения центра орграфа G

Пусть C – матрица стоимостей орграфа G.

Применить процедуру Floyd к матрице C для вычисления матрицы A, содержащей все кратчайшие пути орграфа G.

Найти максимальное значение в каждом столбце i матрицы A. Это значение есть эксцентриситет вершины i.

Найти вершину с наименьшим эксцентриситетом. Это и есть центр орграфа G.

Алгоритм Прима

void Prim (граф G; множество ребер T);

множество вершин U;

вершина u, v;

```

{
    T= ∅;
    U= {1};
    while (U ≠ V)
    {
        нахождение ребра (u, v) наименьшей стоимости и такого, что u ∈ U и v ∈ V \ U;
        T= T ∪ {(u, v)};
        U = U ∪ {v}
    }
}

```

Алгоритм Крускала

Пусть G=(V,E) связный граф со множеством вершин V={1, 2, ..., n} и функцией стоимостей c, определенной на множестве ребер E.

Построение остовного дерева минимальной стоимости для графа G начинается с графа T=(V,∅), состоящего только из n вершин графа G и не имеющего ребер. Таким образом каждая вершина является связной компонентой (сама с собой). В процессе выполнения алгоритма

имеется набор связных компонент, постепенно объединяя которые формируется остовное дерево.

При построении связных компонент поочередно проверяются ребра из множества E в порядке возрастания их стоимости. Если очередное ребро связывает две вершины из разных компонент, тогда оно добавляется в граф T . Если это ребро связывает две вершины из одной компоненты, то оно отбрасывается, так как его добавление в связную компоненту приведет к образованию цикла. Когда все вершины графа G будут принадлежать одной компоненте, построение остовного дерева минимальной стоимости T для этого графа заканчивается.

Задания для самостоятельной работы

1. Реализовать алгоритм Флойда.
2. Реализовать алгоритм Дейкстры.
3. Реализовать алгоритм Прима.
4. Реализовать алгоритм Крускала.
5. Реализовать алгоритм поиска центра графа.
6. Реализовать алгоритм определения сильной связности.

Лабораторная работа №23 **Контейнерные классы. Вектора**

Цель работы:

- освоить основные способы обработки контейнерного класса вектор.

Содержание лабораторной работы

Для использования вектора необходимо включить заголовочный файл:

```
#include <vector>
```

Вектор можно описывать как обычный массив

```
Vector < int > a(10);
```

что аналогично определению массива встроенного типа:

```
int a[ 10 ];
```

Пример. Вычисление суммы вектора

```
#include <iostream>
#include <vector>
using namespace std;
int main()
{
    vector <int> a(5);
    int s=0;
    for(int i=0;i<5;i++)
        cin>>a[i];
    for(int i=0;i<5;i++)
        s+=a[i];
    cout<<s;
    return 0;
}
```

Мы можем узнать размерность вектора, используя функцию `size()`, и проверить, пуст ли вектор, с помощью функции `empty()`.

```
#include <iostream>
#include <vector>
using namespace std;
void print_vector (vector<int> b)
```

```

{
    if ( b.empty() )
        return;
    for ( int i=0; i< b.size(); ++i )
        cout << b[i] << ' ';
}
int main()
{
    vector<int> a(5);
    int s=0;
    for(int i=0;i<5;i++)
        cin>>a[i];
    print_vector(a);
    return 0;
}

```

Задания для лабораторной работы

1. Дан вектор вещественных чисел. Если он не симметричен, то поменять местами в нем первый отрицательный элемент и максимальный (считается, что все элементы в массиве различны).
2. Дан вектор целых чисел. Найти наименьший элемент, индекс которого является числом Фибоначчи.
3. Дан вектор целых чисел. Найти элемент, сумма цифр которого максимальна.
4. Дан вектор целых чисел. Поменять порядок следования элементов на обратный, а в каждом числе поменять порядок следования цифр.
5. Дан вектор целых чисел. Если в векторе хотя бы одна пара элементов образует дружественные числа, переставить эти числа в начало вектора.
6. Дан вектор целых чисел. Найти наименьшее совершенное число, индекс которого кратен трем.
7. Дан вектор вещественных чисел. Если он является симметричным, то найти сумму дробных частей элементов (число после запятой) положительных чисел вектора.
8. Дан вектор целых чисел. Найти количество простых элементов, до и после которых располагаются числа Фибоначчи.

Задания для самостоятельной работы

1. Дан вектор вещественных чисел. Если он не упорядочен, то поменять местами в нем последний положительный элемент и минимальный (считается, что все элементы в векторе различны).
2. Даны действительные числа $a_1, a_2, \dots, a_n$. Если в результате замены отрицательных элементов последовательности $a_1, a_2, \dots, a_n$ их квадратами элементы будут образовывать неубывающую последовательность, то получить сумму элементов исходной последовательности, в противном случае каждый пятый элемент заменить средним арифметическим отрицательных чисел.
3. Дан вектор целых чисел. Найти среднее арифметическое простых элементов вектора.
4. Даны числа $a_1, a_2, \dots, a_n$. Переставить элементы последовательности $a_1, a_2, \dots, a_n$ так, чтобы сначала расположились все неотрицательные элементы, а потом все отрицательные. Порядок как среди неотрицательных элементов, так и среди

- отрицательных, должен быть сохранен прежним. Затем переставить элементы так, чтобы сначала шли четные элементы, затем нечетные.
5. Даны действительные числа $a_1, a_2, \dots, a_n$. Оставить без изменения последовательность $a_1, a_2, \dots, a_n$, если она упорядочена по не убыванию или по не возрастанию, в противном случае удалить из последовательности те элементы, порядковые номера которых кратны четырем, сохранив прежним порядок оставленных элементов.
 6. Дано: действительные числа $a_1, \dots, a_n$, p , натуральное число k ($a_1 \leq a_2 \leq \dots \leq a_n$, $k < n$). Удалить из $a_1, \dots, a_n$ элемент с номером k (т.е. a_k) и вставить элемент, равный p , чтобы не нарушилась упорядоченность.

Лабораторная работа №24 Контейнерные классы. **Stack, set и map**

Цель работы:

- освоить основные способы обработки классов стек, set и map;

Содержание лабораторной работы

Контейнер стека stack позволяет реализовать структуру данных, основанную на принципе LIFO (Last In First Out). Для работы с контейнером **stack** необходимо подключить соответствующую библиотеку:

```
#include <stack>
```

Объявить переменную типа **stack** можно с помощью команды вида

```
stack <тип_элемента> <имя_стека>;
```

Например,

```
stack <int> st;
```

позволяет объявить стек **st**, состоящий из целых значений.

В контейнере **stack** есть predefined методы для работы со стеком. Рассмотрим эти методы:

push() позволяет добавить элемент на верхушку стека;

pop() позволяет извлечь элемент с верхушки стека, при этом сам элемент удаляется;

top() позволяет получить элемент, расположенный на верхушке стека, при этом сам элемент остается в стеке;

size() возвращает количество элементов в стеке;

empty() возвращает **true**, если стек пуст и **false**, если стек не пуст.

Для использования контейнера **set** необходимо подключить заголовочный файл **<set>**

```
#include <set>
```

Для создания пустого множества можно воспользоваться конструкцией вида

```
set <тип> <имя>;
```

Для работы с размером и объемом **set** определены следующие методы:

empty() возвращает **true**, если множество пусто и **false** в противном случае;

size() возвращает текущее количество элементов в множестве;

max_size() возвращает максимально возможное количество элементов в множестве;

Методы, которые позволяют модифицировать множество:

clear() очищает множество;

insert() позволяет добавить элементы;

erase() позволяет удалить элементы; **swap()** обменивает содержимое двух множеств.

К наиболее важному аспекту работы с `set` относится поиск элементов, который осуществляется очень эффективно, поэтому `set` имеет несколько собственных методов поиска:

`count()` возвращает количество элементов, соответствующих определенному ключу;

`find()` находит элемент с конкретным ключом. Возвращает константный итератор позиции элемента или итератор `end()`, если таковой не найден;

`lower_bound()` возвращает итератор на первый элемент, который меньше, чем определенный ключ;

`upper_bound()` возвращает итератор на первый элемент, который больше, чем определенный ключ.

Пример 1. Дана последовательность целых чисел, оканчивающаяся нулем. Вывести в начале положительные значения, отсортированные по возрастанию, а затем – отрицательные, отсортированные по убыванию.

```
#include <iostream>
#include <set>
using namespace std;
int main() {
    setlocale(LC_ALL, "RUSSIAN");
    cout << "Введите элементы (признак завершения 0)\n";
    int x;
    set <int> positive, negative;
    for (cin >> x; x != 0; cin >> x)
        if (x > 0)
            positive.insert(x);
        else
            negative.insert(x);
    cout << "Ответ:" << endl;
    for (set <int>::iterator it = positive.begin(); it !=
positive.end(); ++it)
        cout << *it << ' ';
    for (set <int>::reverse_iterator it=negative.rbegin(); it != negative.rend(); ++it)
        cout << *it << ' ';
    cout << endl;
    system("pause");
    return 0;
}
```

Контейнер `map` (карта, словарь, ассоциативный массив) так же, как и множество, реализован на основе сбалансированного дерева двоичного поиска. Однако его отличием является то, что в словаре хранится набор элементов, состоящий из пар «ключ – значение», при этом ключи должны быть уникальны в пределах словаря и иметь упорядоченный тип.

Для работы с классом `map` необходимо подключить соответствующий заголовочный файл

```
#include <map>
```

Объекты класса `map` можно получить с помощью следующего описания:

```
map <<тип ключа>, <тип значения> > <имя>;
```

Например,

```
map<string, int> ar;
```

Для доступа к элементам словаря можно использовать [], при этом в качестве индекса указывается значение ключа. Если же элемента с таким ключом нет в словаре, то он будет создан со значением, равным нулю. Второй способ получить доступ к элементам словаря – использование метода at(), где в качестве параметра передается значение ключа. Помимо указанных способов доступа можно использовать итераторы для доступа к элементам словаря. С помощью итератора можно обратиться к ключу (it->first) и к значению (it->second).

Методы для работы со словарем являются такими же, как и методы для работы с множествами.

Пример 2. Дано n целых чисел ($n \leq 1000000$). Найти наиболее часто встречающееся число.

```
#include <iostream>
#include <map>
using namespace std;
int main() {
    setlocale(LC_ALL, "RUSSIAN");
    int n;
    cin >> n;
    map <long, long> m;
    for (long i = 0; i < n; ++i) {
        long x;
        cin >> x;
        ++m[x];
    }
    if (n == 0) {
        cout << "не найдено";
        return 0;
    }
    pair<long, long> max = make_pair(m.begin()->first, m.begin()->second);
    for (map <long, long>::iterator it = m.begin(); it != m.end(); ++it) {
        if (max.second < it->second)
            max = make_pair(it->first, it->second);
    } cout << "значение " << max.first << " встречается " << max.second << " раз(a)" << endl;
    system("pause");
    return 0;
}
```

В данном примере значение последовательности определяет ключ словаря m, а количество повторений – его значение.

Pair является переменной, объединяющей в себе два значения, которые могут быть одного или разных типов. Например, чтобы создать переменную типа pair, содержащую два поля типа int нужно использовать следующее объявление:

```
pair <int, int> a;
```

Теперь переменная a будет хранить в себе два значения типа int. Получить доступ к первому значению можно при помощи записи a.first, а ко второму значению — a.second, они называются *полями* переменной. Каждому полю можно присваивать независимые значения, например, так:

```
a.first = 1;
a.second = 2;
```

Присвоить одновременно два значения обоим полям пары можно при помощи функции `make_pair`, принимающей два аргумента и возвращающей значение типа `pair` с соответствующими полями:

```
a = make_pair(1, 2);
```

Между тем с переменной типа `pair` можно работать и как с единым целым, например, если объявить две переменные типа `pair` с одинаковыми типами полей, то можно одной переменной присвоить значение другой переменной:

```
pair<int, int> a, b;
```

```
...
```

```
b = a;
```

Но при вводе-выводе переменных, типа `pair` нельзя выводить или вводить значение переменной одной командой, то есть нельзя сразу же вывести два значения типа:

```
cin >> a;
```

```
cout << a;
```

Вместо этого нужно отдельно считывать или выводить каждое поле пары.

Можно создавать пару с полями разных типов, например, чтобы поле `first` имело тип `double`, а поле `second` — `string`, нужно использовать запись `pair<double, string>`.

С парой, как с целым (то есть одновременно с двумя полями структуры) можно выполнять следующие операции: `=`, `==`, `!=`, `<`, `<=`, `>`, `>=`, а также использовать в качестве аргумента функции `swar`. При использовании операций сравнения (типа меньше, больше и т.д.) пары сравниваются в лексикографическом порядке, то есть сначала по полю `first`, а если поля `first` равны, то по полю `second`.

Вектор, составленный из структур `pair` можно сортировать, при этом используется сравнение типа `<`, то есть пары упорядочиваются по первому полю, затем — по второму полю. Объявить вектор из пар можно следующим образом:

```
vector< pair<int, int> > a
```

Обратите внимание на пробел между закрывающими угловыми скобками: он обязателен, иначе компилятор будет считать это операцией `>>`.

Задания для лабораторной работы

SET, MAP:

1. Во входной строке записана последовательность чисел через пробел. Для каждого числа выведите слово **YES** (в отдельной строке), если это число ранее встречалось в последовательности или **NO**, если не встречалось.
2. Во входном файле задан набор слов и целых чисел, разделенных пробелами. Найти все слова, встречающиеся столько же раз, сколько первое число.
3. Анаграммы – это слова, у которых те же буквы в том же количестве, но они располагаются в другом порядке. Исключить из массива строк слова-анаграммы.

СТЕК:

1. Поменять местами первый и последний элементы стека.
2. Развернуть стек, т.е. сделать "дно" стека вершиной, а вершину – "дном"
3. Удалить элемент, находящийся в середине стека, если число элементов нечетное, или 2 средних элемента, если число элементов четное.
4. Удалить каждый второй элемент стека
5. Вставить символ `*` в середину стека, если число элементов четное, или после среднего элемента, если число элементов нечетное.
6. Найти минимальный элемент и вставить после него 0.

Задания для самостоятельной работы

СТЕК:

7. Найти максимальный элемент и вставить после него 0
8. Удалить минимальный элемент.
9. Удалить все элементы, равные первому.
10. Удалить все элементы, равные последнему.
11. Удалить максимальный элемент.
12. Найти минимальный элемент и удвоить его.

Лабораторная работа №25 Контейнерные классы. Списки

Цель работы:

- освоить основные способы обработки класса список.

Содержание лабораторной работы

Контейнер `list` представляет двухсвязный список. Для его использования необходимо подключить заголовочный файл `list`.

```
list<int> list1;           // пустой список
list<int> list2(5);        // список list2 состоит из 5
чисел, каждый элемент имеет значение по умолчанию
list<int> list3(5, 2);     // список list3 состоит из 5
чисел, каждое число равно 2
```

Для получения размера списка можно использовать функцию **size()**:

```
list<int> numbers = { 1, 2, 3, 4, 5 };
int size = numbers.size(); // 5
```

Функция **empty()** позволяет узнать, пуст ли список. Если он пуст, то функция возвращает значение `true`, иначе возвращается значение `false`:

```
list<int> numbers = { 1, 2, 3, 4, 5 };
if (numbers.empty())
    cout << "The list is empty" << endl;
else
```

```
    cout << "The list is not empty" << endl;
```

С помощью функции **resize()** можно изменить размер списка. Эта функция имеет две формы:

- **resize(n)**: оставляет в списке `n` первых элементов. Если список содержит больше элементов, то он усекается до первых `n` элементов. Если размер списка меньше `n`, то добавляются недостающие элементы и инициализируются значением по умолчанию

- **resize(n, value)**: также оставляет в списке `n` первых элементов. Если размер списка меньше `n`, то добавляются недостающие элементы со значением `value`

```
list<int> numbers = { 1, 2, 3, 4, 5, 6 };
numbers.resize(4); // оставляем первые четыре элемента -
numbers = {1, 2, 3, 4}
```

```
numbers.resize(6, 8); // numbers = {1, 2, 3, 4, 8, 8}
```

Функция **assign()** позволяет заменить все элементы списка определенным набором. Она имеет следующие формы:

- **assign(il)**: заменяет содержимое контейнера элементами из списка инициализации `il`

- **assign(n, value)**: заменяет содержимое контейнера `n` элементами, которые имеют значение `value`

• **assign(begin, end)**: заменяет содержимое контейнера элементами из диапазона, на начало и конец которого указывают итераторы **begin** и **end**

```
list<int> numbers = { 1, 2, 3, 4, 5 };
numbers.assign({ 21, 22, 23, 24, 25 }); // numbers = { 21, 22,
23, 24, 25 }
numbers.assign(4, 3); // numbers = {3, 3, 3, 3}
list<int> values = { 6, 7, 8, 9, 10, 11 };
auto start = ++values.begin(); // итератор указывает на второй
элемент из values
auto end = values.end();
numbers.assign(start, end); // numbers = { 7, 8, 9, 10, 11 }
```

Функция **swap()** обменивает значениями два списка:

```
list<int> list1 = { 1, 2, 3, 4, 5 };
list<int> list2 = { 6, 7, 8, 9 };
list1.swap(list2);
// list1 = { 6, 7, 8, 9 };
// list2 = { 1, 2, 3, 4, 5 };
```

Для добавления элементов в контейнер **list** применяется ряд функций.

- **push_back(val)**: добавляет значение **val** в конец списка
- **push_front(val)**: добавляет значение **val** в начало списка
- **emplace_back(val)**: добавляет значение **val** в конец списка
- **emplace_front(val)**: добавляет значение **val** в начало списка
- **emplace(pos, val)**: вставляет элемент **val** на позицию, на которую указывает итератор **pos**. Возвращает итератор на добавленный элемент

- **insert(pos, val)**: вставляет элемент **val** на позицию, на которую указывает итератор **pos**, аналогично функции **emplace**. Возвращает итератор на добавленный элемент

- **insert(pos, n, val)**: вставляет **n** элементов **val** начиная с позиции, на которую указывает итератор **pos**. Возвращает итератор на первый добавленный элемент. Если **n = 0**, то возвращается итератор **pos**.

- **insert(pos, begin, end)**: вставляет начиная с позиции, на которую указывает итератор **pos**, элементы из другого контейнера из диапазона между итераторами **begin** и **end**. Возвращает итератор на первый добавленный элемент. Если между итераторами **begin** и **end** нет элементов, то возвращается итератор **pos**.

- **insert(pos, values)**: вставляет список значений **values** начиная с позиции, на которую указывает итератор **pos**. Возвращает итератор на первый добавленный элемент. Если **values** не содержит элементов, то возвращается итератор **pos**.

```
list<int> numbers = { 1, 2, 3, 4, 5 };
numbers.push_back(23); // { 1, 2, 3, 4, 5, 23 }
numbers.push_front(15); // { 15, 1, 2, 3, 4, 5, 23 }
numbers.emplace_back(24); // { 15, 1, 2, 3, 4, 5, 23, 24 }
}
numbers.emplace_front(14); // { 14, 15, 1, 2, 3, 4, 5, 23,
24 }
```

Для удаления элементов из контейнера **list** могут применяться следующие функции:

- **clear(p)**: удаляет все элементы
- **pop_back()**: удаляет последний элемент
- **pop_front()**: удаляет первый элемент

• **erase(p)**: удаляет элемент, на который указывает итератор p. Возвращает итератор на элемент, следующий после удаленного, или на конец контейнера, если удален последний элемент

• **erase(begin, end)**: удаляет элементы из диапазона, на начало и конец которого указывают итераторы begin и end. Возвращает итератор на элемент, следующий после последнего удаленного, или на конец контейнера, если удален последний элемент

```
list<int> numbers = { 1, 2, 3, 4, 5 };
numbers.pop_front();    // numbers = { 2, 3, 4, 5 }
numbers.pop_back();     // numbers = { 2, 3, 4 }
numbers.clear();        // numbers = {}

numbers = { 1, 2, 3, 4, 5 };
auto iter = numbers.cbegin(); // указатель на первый
элемент
numbers.erase(iter);        // удаляем первый элемент
// numbers = { 2, 4, 5, 6 }

numbers = { 1, 2, 3, 4, 5 };
auto begin = numbers.begin(); // указатель на первый
элемент
auto end = numbers.end();      // указатель на последний
элемент
numbers.erase(++begin, --end); // удаляем со второго
элемента до последнего
//numbers = {1, 5}
```

Задания для лабораторной работы

1. Даны 2 кольцевых списка, содержащие перечень товаров, производимых концернами BOSH и FILIPS. Создать кольцевой список товаров, выпускаемых как одной, так и другой фирмой.
2. Даны 2 кольцевых списка, содержащие фамилии футболистов основного состава команды и запасного. Произвести K замен. Число пересчета для основного состава - E, для запасного – K (E-й игрок заменяется K-м).
3. Даны 2 кольцевых списка, содержащие перечень товаров и фамилии покупателей. Каждый N-й покупатель покупает M-й товар. Вывести список покупок.
4. Даны 2 кольцевых списка, содержащие наименования товаров, выпускаемых фирмами SONY и SHARP. Создать список конкурирующих между собой товаров.
5. Дан кольцевой список, содержащий фамилии студентов группы. Разбить его на два кольцевых списка, поместив во второй список – каждого третьего студента.
6. Даны 2 кольцевых списка, содержащие перечень товаров. Объединить их в один кольцевой список. Число пересчета для первого списка – 4, для второго – 3 (помещаем в объединенный список 4-й элемент первого, затем 3-й элемент второго списка и т.д.)
7. Задача Джозефуса: n воинов из одного войска убивают каждого m-го из другого. Требуется определить номер k начальной позиции воина, который должен будет остаться последним.
8. Фирма должна послать в командировку N сотрудников в M городов. Даны 2 кольцевых списка, содержащие фамилии всех сотрудников фирмы и города.

Выбрать откомандированных сотрудников и вывести распределение: сотрудник-город. Число пересчета для сотрудников K1, для городов K2.

Задания для самостоятельной работы

1. Дан кольцевой список, содержащий 20 фамилий игроков футбольной команды. Разбить игроков на 2 группы по 10 человек. Во вторую группу попадает каждый 3-й человек.
2. Даны 2 кольцевых списка, содержащие фамилии спортсменов двух фехтовальных команд. Произвести жеребьевку. В первой команде выбирается каждый n -й игрок, а во второй - каждый m -й.
3. Даны 2 кольцевых списка, содержащие фамилии участников лотереи и наименования призов. Выиграет N человек (каждый K -й). Число для пересчета призов – m (выбирается каждый m -й приз). Вывести фамилии выигравших и их призы.

Лабораторная работа №26

Контейнерные классы. Очереди

Цель работы:

- освоить основные способы обработки класса очереди.

Содержание лабораторной работы

Контейнер очереди `queue` позволяет реализовать структуру данных, основанную на принципе FIFO (First In First Out). Для работы с контейнером `queue` необходимо подключить соответствующую библиотеку:

```
#include <queue>
```

Объявить переменную типа `queue` можно с помощью команды вида

```
queue <тип_элемента> <имя_очереди>;
```

Например,

```
queue <int> q;
```

позволяет объявить очередь `q`, состоящую из целых значений.

В контейнере `queue` предопределены следующие методы для работы с очередью:

`push()` позволяет добавить элемент в конец очереди;

`pop()` позволяет удалить элемент из начала очереди;

`front()` позволяет получить элемент, расположенный в начале очереди, при этом сам элемент остается в очереди;

`back()` позволяет получить элемент, расположенный в конце очереди, при этом сам элемент остается в очереди;

`size()` возвращает количество элементов в очереди;

`empty()` возвращает `true`, если очередь пуста и `false`, если очередь не пуста.

Задания для лабораторной работы

1. Создайте очередь и добавьте в неё несколько элементов. Затем удалите каждый второй элемент из очереди.
2. Создайте очередь и добавьте в неё несколько чисел. Затем отсортируйте элементы очереди в порядке убывания.

3. Создайте очередь и добавьте в неё целые числа. Затем найдите среднее значение элементов очереди и поставьте первым элементом.
4. Создайте очередь и заполните её элементами последовательности чисел Фибоначчи. Затем удалите все числа, которые делятся на 3.
5. Создайте очередь, добавьте в неё несколько чисел, а затем измените последний элемент на значение суммы всех элементов в очереди.
6. Создайте очередь и добавьте в неё несколько целых чисел. Измените очередь так, чтобы после каждого элемента следующим элементом был квадрат этого числа.
7. Создайте очередь и заполните её случайными числами. После этого отсортируйте первую половину элементов в порядке возрастания, а вторую половину - в порядке убывания.
8. Создайте очередь из случайных целых чисел. Разделите ее на две очереди: в первой – четные числа, во второй – нечетные.

Задания для самостоятельной работы

1. Создайте очередь и добавьте в неё элементы "apple", "banana", "orange". Затем соедините строку "fruit" с каждым элементом очереди и отобразите результат.
2. Создайте очередь и добавьте в неё несколько чисел. Затем отобразите содержимое очереди в обратном порядке, не изменяя саму очередь.
3. Создайте очередь, добавьте в неё несколько строк. Если количество элементов в очереди четное, то удалите два последних элемента, если нечетное - один.
4. Создайте очередь и заполните её случайными числами. Найдите наименьшее и наибольшее значение в очереди.